Optimising Attention on a CPU

CS6886 – Assignment 5 November 6, 2026

Q1. Machine

Field	Value
CPU model	Intel Core i5-12500H
Microarchitecture	Alder Lake: Golden Cove / Gracemont
Physical / logical cores	12 physical / 16 logical
Performance (P) / efficiency (E) cores	4 P cores (2 threads each) / 8 E cores (1 each)
Base clock	P: 2.5 GHz; E: 1.8 GHz
Maximum single-core clock	P: 4.5 GHz; E: 3.3 GHz
All-core clock under load	About P: 3.5 GHz; E: 3.1 GHz, sampled during memory test
SIMD instruction set and width	AVX2 + FMA; 256 bits; 8 FP32 values
FMA units per core	P: 2 × 256-bit; E: 2 × 128-bit
Vector registers	16 YMM registers per hardware thread
L1 data cache, per core	P: 48 KiB; E: 32 KiB
L2 cache and sharing	P: 1.25 MiB/core; E: 2 MiB/four-core group
L3 cache and sharing	18 MiB shared by all cores
Cache line size	64 bytes
RAM size	16 GiB (17.18 GB); OS usable: 15.24 GiB
RAM type, speed, channels	DDR4; 3200 MT/s; 2 channels, each 8 GiB
Operating system	Ubuntu 24.04.4 LTS; Linux 7.0.0-42-generic
Compiler and version	GCC 13.3.0; OpenMP through libgomp
Peak FP32, one core	P: 144 GFLOP/s; E: 52.8 GFLOP/s
Peak FP32, all cores	998.4 GFLOP/s maximum-clock upper bound
Memory bandwidth	51.2 GB/s theoretical; 21.16 one-core / 31.29 best measured
Ridge point, one / all cores	6.81 / 31.91 FLOP/byte, using measured bandwidth

Calculations. One FMA counts as two operations; 256 bits hold eight FP32 values.

$$\begin{aligned}
\text{One P core:} &\quad 4.5 \times 8 \times 2 \times 2 = 144 \text{ GFLOP/s} \\
\text{One E core:} &\quad 3.3 \times 8 \times 1 \times 2 = 52.8 \text{ GFLOP/s} \\
\text{All cores:} &\quad 4 \times 144 + 8 \times 52.8 = 998.4 \text{ GFLOP/s} \\
\text{At load clocks:} &\quad 4 \times 3.5 \times 32 + 8 \times 3.1 \times 16 = 844.8 \text{ GFLOP/s} \\
\text{Memory bandwidth:} &\quad 3200 \times 2 \times 8/1000 = 51.2 \text{ GB/s} \\
\text{Measured ridge points:} &\quad 144/21.16 = 6.81; \quad 998.4/31.29 = 31.91 \text{ FLOP/byte} \\
\text{Theoretical ridge points:} &\quad 144/51.2 = 2.81; \quad 998.4/51.2 = 19.50 \text{ FLOP/byte}
\end{aligned}$$

Maximum clocks need not hold on all cores together. Measured bandwidth uses a 192 MiB memory test, counting two reads and one write (12 bytes/element).

Q2. Time and scaling

Doubling N at d=64 increases work four times; naive time rises 3.99× and then 5.58×. Doubling d at N=4096 doubles work, but time rises 3.31× as more data exceeds the caches. About 72% of naive time goes to the final multiply, 24% to scores and 4% to softmax.

Time in ms (speedup over naive). Column labels are N / d.

Variant	1024 / 64	2048 / 64	4096 / 64	4096 / 128
naive	52.810 (1.00×)	210.480 (1.00×)	1175.305 (1.00×)	3889.333 (1.00×)
loops	12.133 (4.35×)	48.291 (4.36×)	212.877 (5.52×)	373.320 (10.42×)
tiled	11.094 (4.76×)	43.592 (4.83×)	185.517 (6.34×)	268.113 (14.51×)
fused	12.119 (4.36×)	49.218 (4.28×)	212.409 (5.53×)	360.218 (10.80×)
best	2.816 (18.75×)	11.186 (18.82×)	45.193 (26.01×)	82.815 (46.96×)

GFLOP/s (percentage of the 144 GFLOP/s one-core peak).

Variant	1024 / 64	2048 / 64	4096 / 64	4096 / 128
naive	5.08 (3.5%)	5.10 (3.5%)	3.65 (2.5%)	2.21 (1.5%)
loops	22.12 (15.4%)	22.23 (15.4%)	20.18 (14.0%)	23.01 (16.0%)
tiled	24.20 (16.8%)	24.63 (17.1%)	23.15 (16.1%)	32.04 (22.2%)
fused	22.15 (15.4%)	21.82 (15.2%)	20.22 (14.0%)	23.85 (16.6%)
best	95.33 (66.2%)	95.99 (66.7%)	95.04 (66.0%)	103.72 (72.0%)

Median of three runs, each with one warm-up and five timed calls; CPU 0, GCC `-O3 -march=native`. Times include allocations; GFLOP/s counts $4N^2d$ operations.

Q3. Four variations

a) Loops: Change the loop order and transpose K so nearby values are read together. The compiler processes several values at once. Repeated updates and softmax still cost time.

b) Tiled: Work on small groups of queries and keys, keeping partial sums close to the CPU. This reuses data; at 4096/64, time falls from 212.877 to 185.517 ms. Softmax and output updates remain costly.

c) Fused: Finish one row before starting the next. Score storage falls from 64 MiB to 16 KiB at N=4096. Time is 212.409 ms: lower storage alone does not improve reuse across rows.

d) Best (FlashAttention): Update softmax and output one key block at a time. When a larger score appears, rescale earlier contributions. Keep only an 8 KiB score tile, then divide the output by the final exponential sum.

All checks pass. Maximum relative output error: `6.3e-06`; allowed: 10^{-4}.

Q4. OpenMP

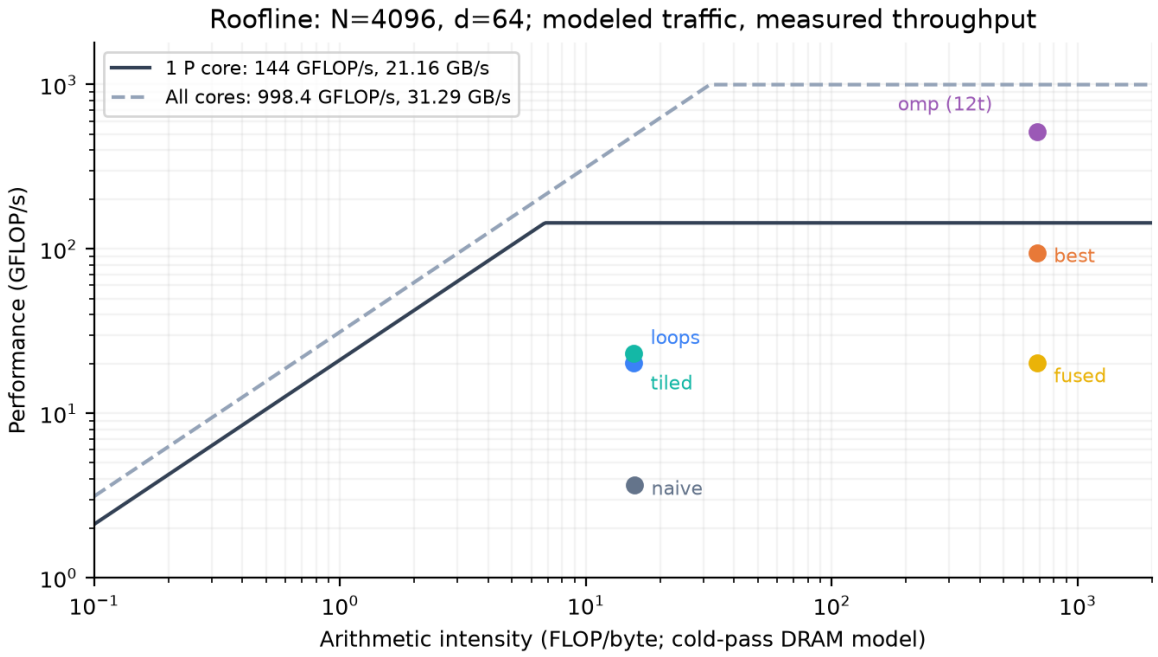
For 4096/64, 12 threads cut time from 45.622 to 8.357 ms: 5.46× faster, at 45.5% efficiency. Workers handle separate query blocks; gains shrink as cores share calculation and cache resources.

Threads	1024/64 ms	2048/64 ms	4096/64 ms	4096/128 ms	S	Efficiency
1	2.737	11.244	45.622	82.465	1.00	100.0%
2	1.515	5.805	23.452	43.515	1.95	97.3%
4	0.890	3.119	13.676	24.713	3.34	83.4%
6	0.761	2.848	11.741	21.566	3.89	64.8%
8	0.805	2.728	10.070	18.898	4.53	56.6%
12	0.663	2.199	8.357	15.856	5.46	45.5%
16	0.691	2.464	9.044	16.949	5.04	31.5%

S and efficiency use 4096/64: S = one-thread time / p-thread time; efficiency = S / p.

Q5. Roofline

The plot compares measured speed with compute and memory limits at N=4096, d=64. Moving and reusing data better explains the gains from loops and tiling; fusion alone is slower than tiling. Best uses online softmax; OpenMP reaches 513.94 GFLOP/s using 12 threads.



Variant	Intensity (FLOP/B)	Measured GFLOP/s
naive	15.754	3.65
loops	15.634	20.18
tiled	15.634	23.15
fused	682.667	20.22
best	682.667	95.04
omp (12 threads)	682.667	513.94

Intensity is work divided by estimated bytes transferred from RAM: naive uses $4N^2d/(16Nd{+}16N^2)$; loops/tiled use $4N^2d/(24Nd+16N^2)$; fused/best/OpenMP use $N/6$. These estimates count four score-matrix passes and packed K, assume repeated K/V reads stay in cache, and omit extra writes caused by cache allocation. Traffic is modeled, not measured.

Sources: local `lscpu`, cache/clock files and RAM information; Intel CPU specifications; uops.info; FlashAttention. Full results and repeat timings are in `results.csv` and `measurements.json`.